

Practical Implementation Tutorial

Step-by-Step Guide: Implementing Safety & Code Formatting Rules in .agents/AGENTS.md

Tutorial Overview: This practical manual walks you through authoring, structuring, and verifying an .agents/AGENTS.md rule file in Google Antigravity. Unlike skills, rules are **ambient and always-on** within their directory subtree, providing enforceable guardrails against secret leaks, destructive commands, and formatting inconsistencies.

Step 1: Create the .agents Directory & Rule File

Place the rule file at the repository root inside .agents/AGENTS.md (or directly at /AGENTS.md):

```
# In your project root terminal:
mkdir -p .agents
touch .agents/AGENTS.md
```

Step 2: Define Non-Negotiable Safety Protocols

Safety guardrails must be phrased using clear, imperative constraints. Key categories include:

- **Credential Redaction:** 'Never log, display, or commit API keys, secrets, JWT tokens, or .env contents.'
- **Destructive Operations:** 'Never execute destructive commands (e.g. `rm -rf`, `git reset --hard`, `DROP TABLE`) without explicit user review.'
- **Path Confinement:** 'Confine file modifications strictly to the project workspace root.'

Step 3: Define Language-Specific Formatting & Quality Standards

Specify exact linter, formatter, and type-system expectations for your stack:

- **TypeScript / JavaScript:** Strict typing (no any), ESLint/Prettier compliance, preference for async/await.
- **Python:** PEP 8 compliance, Black / Ruff formatting, strict type hints on function signatures.
- **Docstring Preservation:** Mandate preserving existing comments and writing JSDoc / docstrings on new public exports.

Step 4: Complete Production Template for .agents/AGENTS.md

Copy and adapt this production template into .agents/AGENTS.md:

```
# Project Operational Rules & Safety Protocols

## 1. Safety & Security Guardrails
- Credential Protection: Never log, display, or hardcode API keys, secrets, or .env files.
- Destructive Commands: Never execute rm -rf, git reset --hard, or DROP DATABASE without confirmation.
- Workspace Boundary: Confine file reads and writes strictly to the project workspace root.

## 2. Code Formatting & Quality Standards
- TypeScript: Enforce ESLint/Prettier formatting; require strict return types; avoid any.
- Python: Adhere to PEP 8 with Black/Ruff formatting and strict type hints (typing).
- Documentation: Preserve all existing comments; provide JSDoc/docstrings on exported functions.

## 3. Automated Verification Mandate
- Run project test suites (npm test / pytest) and linters before completing any task.

## 4. Deliverable Format Policy
- All comprehensive explanations and answers must be delivered in professional PDF format.
```

Step 5: Best Practices & Verification

- **Commit to Git:** Check .agents/AGENTS.md into version control so the whole team benefits from identical guardrails.
- **Keep Rules Lean:** Avoid general coding textbook explanations; focus on project-specific constraints and invariants.

- **Hierarchical Layering:** In monorepos, add localized AGENTS.md in subdirectories (e.g. frontend/AGENTS.md) for localized UI/backend rules.